Taller 4: Manipulación de Datos en R III

Estadística II

M. Constanza Ayala

maria.ayala@uv.cl

Agenda una reunión aquí

Escuela de Sociología - Universidad de Valparaíso

31 de marzo, 2026

Pasemos lista mientras descargan los materiales y dejamos todo listo ✨

📂 **Paso 1:** Abrimos nuestro proyecto con doble clic en el archivo `.Rproj`

- Si no tienen proyecto, creamos uno: - `File > New Project > New Directory > New Project` - Creamos carpetas: `data`, `scripts`, `figures`, `output`

💾 **Paso 2:** Movemos el archivo **"Script Taller 4 EstadisticaII.R"** a la carpeta `scripts`

📎 **Paso 3:** Verificamos que la base **ELSOC_Wide_2016_2023.RData** y **Base Evaluaciones ELPI III_reducida.RData** esté en la carpeta `data`

> **Todos los materiales los pueden descargar en este link:** Materiales clases

Evaluación 2

Hoy tenemos nuestra **segunda evaluación individual** en R. Debes entregar lo siguiente:

- Proyecto de R (.Rproj).
- El script (.R) que contiene todos los ejercicios individuales solicitados en este taller. Debes crear un script nuevo, no utilices el script de actividades entregado en la clase.
  - En ese script, incluye un título, tu nombre, correo y fecha. Por ejemplo:

```
# Script Ejercicio 2 Estadística II ------------------------------------------
# M. Constanza Ayala (maria.ayala@uv.cl)
# 31-03-2026
```

Plazo de entrega: hoy hasta las 23:59 hrs. en el Aula Virtual.

> En caso de utilizar IA, debe reportarlo en un archivo Word, indicando la herramienta y los prompts usados. El uso no reportado será calificado con nota mínima.

Objetivos de la clase

- Retomar el paquete **`dplyr` para manipulación de datos**

- **Transformar y crear** nuevas variables: `mutate()`

¿Qué vimos la semana pasada en R?

✅ Utilizamos `filter()` para seleccionar filas según condiciones

✅ Utilizamos `select()` para elegir o eliminar columnas

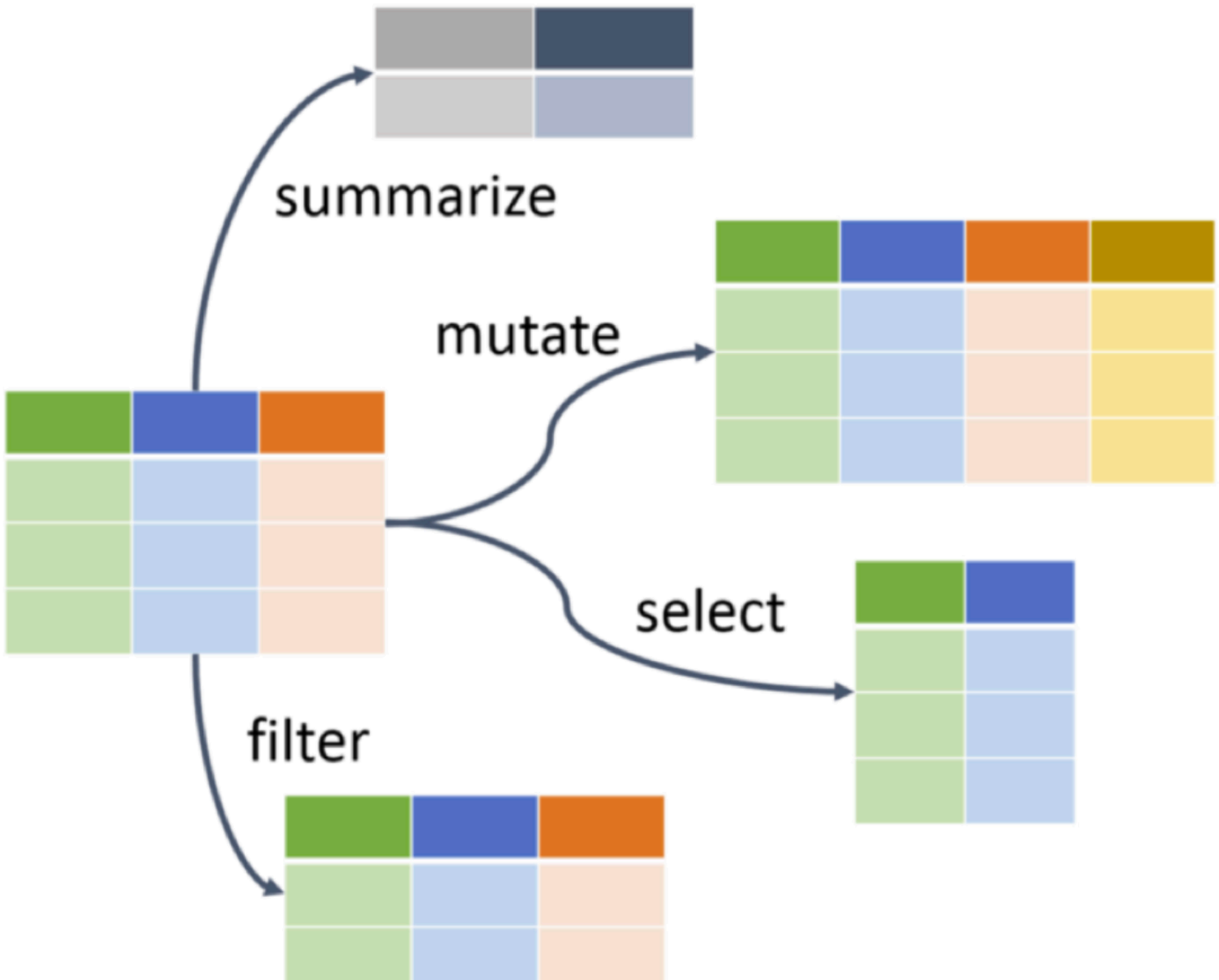
✅ Utilizamos `arrange()` para ordenar filas

💡 Hoy seguimos con la **manipulación de datos** en R.

Actividad 1 para ejercicio

1. Carga el paquete `dplyr` en una sección del script llamada `Paquetes`.
2. Carga la base `ELSOC_Wide_2016_2023.RData` en una sección llamada `Base de datos`.
3. Explora la base con `glimpse()` en una sección llamada `Exploración base de datos`.

Manipulación de datos con dplyr



Generar nuevas variables

MAKE NEW VARIABLES

Apply **vectorized functions** to columns. Vectorized functions take vectors as input and return vectors of the same length as output (see back).

mutate(.data, …, .keep = "all", .before = NULL, .after = NULL**)** Compute new column(s). Also **add_column(), add_count(),** and **add_tally().**
mutate(mtcars, gpm = 1 / mpg)

transmute(.data, …**)** Compute new column(s), drop others.
transmute(mtcars, gpm = 1 / mpg)

rename(.data, …**)** Rename columns. Use **rename_with()** to rename with a function.
rename(cars, distance = dist)

mutate()

- Crea **columnas nuevas** a partir de operaciones sobre columnas existentes
- Las nuevas columnas se agregan al final del dataframe
- Hoy veremos tres usos:
  - Operaciones aritméticas: sumas y promedios
  - Recodificación de valores perdidos con `case_when()`
  - Recodificación de valores o categorías con `case_when()`

Carguemos el paquete y una base de datos

```r
#Paquete
library(dplyr)

#Base de datos
load("data/Base Evaluaciones ELPI III_reducida.RData")

# Exploremos los datos con dplyr
data %>% glimpse() # Vista previa
```

```
Rows: 15,827
Columns: 10
$ folio          <dbl> 100011, 100081, 100091, 100121, 100131, 100151,
100161,…
$ espanel        <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1…
$ sexo           <dbl> 2, 1, 1, 1, 1, 2, 1, 1, 1, 2, 1, 2, 1, 1, 2, 2, 1, 1,
2…
$ nivel_edu_niño <dbl> 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4,
4…
$ edad_mesesr    <dbl> 144, 144, 146, 144, 143, 147, 146, 143, 143, 147, 142,
…
```

Operaciones aritméticas

- La base ELPI incluye tres puntajes de matemáticas: `wm_pb_pa`, `wm_pb_fd` y `wm_pb_cc`.
- Podemos crear una variable de **suma total** y una de **promedio**:

```r
# Suma
data <- data %>%
  mutate(suma_matematica = wm_pb_pa + wm_pb_fd + wm_pb_cc)

summary(data$suma_matematica)
```

```
 Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's 
 0.00   66.00   81.00   82.06   98.00  223.00    5162 
```

```r
# Promedio
data <- data %>%
  mutate(media_matematica = (wm_pb_pa + wm_pb_fd + wm_pb_cc) / 3)

summary(data$media_matematica)
```

```
 Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's 
 0.00   22.00   27.00   27.35   32.67   74.33    5162 
```

> Si alguna de las variables tiene `NA`, el resultado también será `NA`.

¿Qué pasa con los valores perdidos?

Podemos usar `rowSums()` y `rowMeans()` para controlar el tratamiento de los `NA`:

```r
# na.rm = FALSE → si hay NA, el resultado es NA (comportamiento por defecto)
data <- data %>%
  mutate(suma_mat_naF = rowSums(cbind(wm_pb_pa, wm_pb_fd, wm_pb_cc),
                                na.rm = FALSE))

# na.rm = TRUE → suma lo disponible aunque falten datos
data <- data %>%
  mutate(suma_mat_naT = rowSums(cbind(wm_pb_pa, wm_pb_fd, wm_pb_cc),
                                na.rm = TRUE))

summary(data$suma_mat_naF)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's 
   0.00   66.00   81.00   82.06   98.00  223.00    5162 
```

```r
summary(data$suma_mat_naT)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
   0.00    0.00   67.00   55.34   89.00  223.00 
```

> La decisión de usar `na.rm = TRUE` o `FALSE` depende de la lógica del análisis. Ambas opciones son válidas en distintos contextos.

case_when()

- Permite crear variables nuevas según **condiciones lógicas**
- Funciona como una serie de reglas: *si se cumple esta condición, asigna este valor*

```r
datos <- datos %>%
  mutate(nueva_variable = case_when(
    condición_1 ~ valor_1,
    condición_2 ~ valor_2,
    TRUE        ~ valor_si_ninguna_se_cumple
  ))
```

> La línea `TRUE ~ ...` al final actúa como "todos los demás casos". Si no la incluyes, los casos que no cumplan ninguna condición quedarán como `NA`.

Uso 1: recodificar valores perdidos

Algunas bases de datos codifican los valores perdidos con números como –999 o –888. Antes de operar con esas variables, debemos reemplazarlos por NA:

```r
data <- data %>%
  mutate(sexo = case_when(
    sexo == -999 ~ NA_real_,
    sexo == -888 ~ NA_real_,
    TRUE         ~ as.numeric(sexo)
  ))

table(data$sexo, exclude = FALSE)
```

```
   1    2
8095 7732
```

> Para variables de texto usamos `NA_character_`, para numéricas usamos `NA_real_`.

Uso 2: recodificar una variable numérica en grupos

```r
# Clasificamos la edad del niño en valores
summary(data$edad_mesesr)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  11.00   69.00  110.00   98.51  128.00  151.00 
```

```r
data <- data %>%
  mutate(grupo_edad = case_when(
    edad_mesesr < 72  ~ 1,
    edad_mesesr >= 72 ~ 2,
    TRUE              ~ NA_real_
  ))

table(data$grupo_edad, exclude = FALSE)
```

```
   1     2 
4114 11713 
```

Uso 3: recodificar en múltiples categorías

También podemos usar `case_when()` para crear variables con nuevas categorías a partir de una variable existente:

```
table(data$idregion)
```

```
   1    2    3    4    5    6    7    8    9   10   11   12   13   14   15 
 404  745  305  830 1700  865  972 1799  808  839  128  150 5744  274  264 
```

```
data <- data %>%
  mutate(zona = case_when(
    idregion == 13              ~ "Metropolitana",
    idregion %in% c(1:4, 15)  ~ "Norte",
    idregion %in% c(5:7)      ~ "Centro",
    idregion %in% c(8:12, 14) ~ "Sur",
    TRUE                        ~ NA_character_
  ))

table(data$zona, exclude = FALSE)
```

```
      Centro Metropolitana         Norte           Sur 
        3537          5744          2548          3998 
```

Actividad 2 ejercicio (base ELSOC)

1. En una sección llamada `Mutate 1`, recodifica los valores `-999` y `-888` como `NA` en las variables `t02_01_w07`, `t02_02_w07` y `t02_03_w07` (ítems de identidad barrial, ola 2023) usando `mutate()` y `case_when()`. Revisa el resultado con `table(..., exclude = FALSE)` para al menos una de las variables.
2. En una sección llamada `Mutate 2`, crea la variable `media_identidad_barrio`: promedio de los tres ítems anteriores usando `rowMeans()` con `na.rm = TRUE`. Revisa la nueva variable con `summary()`.
3. En una sección llamada `Mutate 3`, crea la variable `grupo_edad` que clasifique `m0_edad_w07` en tres grupos usando `mutate()` y `case_when()`:
   - `"Adulto/a joven"`: entre 18 y 39 años
   - `"Adulto/a"`: entre 40 y 59 años
   - `"Adulto/a mayor"`: 60 años o más
   - Valores fuera de rango → `NA_character_`

   Revisa la nueva variable con `table(..., exclude = FALSE)`.

Buenas prácticas al usar `mutate()`

✅ Usa nombres descriptivos: `media_identidad_barrio`, `grupo_edad`, etc.

✅ Revisa siempre los resultados con `summary()` o `table()`

✅ Recodifica los valores perdidos **antes** de calcular sumas o promedios

✅ Controla el tratamiento de `NA` explícitamente: - En `rowMeans()` / `rowSums()`: usa `na.rm = TRUE/FALSE` - En `case_when()`: incluye `TRUE ~ NA_character_` o `TRUE ~ NA_real_` al final

❓ Preguntas y aclaraciones

💬 **¿Dudas sobre lo visto hoy?**

- Tómense un momento para reflexionar y compartir preguntas sobre el contenido.
- Espacio para responder inquietudes y aclarar conceptos clave.

Resumen clase de hoy

- ✅ Aprendimos a usar `mutate()` para crear nuevas variables
- ✅ Calculamos sumas y promedios con y sin `NA`
- ✅ Recodificamos valores perdidos con `case_when()`
- ✅ Recodificamos categorías con `case_when()`

📅 Próxima sesión (martes)

- Probabilidades y aplicación de pruebas de hipótesis

📚 Sugerencias lecturas para reforzar R

- Wickham & Grolemund. R for Data Science (acceso en línea biblioteca, en español: https://es.r4ds.hadley.nz/)
- AnalizaR Datos Políticos
- YaRrr! The Pirate's Guide to R